
作业三实验报告

王琳睿 241300009 (email: 1637928975@qq.com)

(南京大学 人工智能学院, 南京 210093)

1 不同学习方法介绍

模型学习的是"状态→动作"的映射关系，这属于多类别分类问题。

1.1 随机森林:

随机森林或随机决策森林是一种监督式算法,是一种用于分类、回归和其他任务的集成学习方法,它通过在训练期间创建大量决策树,并综合所有树的结果来工作。对于分类任务,随机森林的输出是大多数树选择的类。对于回归任务,输出是树预测的平均值。随机森林能纠正决策树过度拟合其训练集的习惯。^[1]

算法步骤:

1. 随机森林的核心——两个“随机”:

首先,从原始数据集中有放回地随机抽取样本,形成一个用于训练单棵树的子数据集。这意味着同一个样本可能被抽中多次,也可能一次都没被抽中。而那些没被抽中的样本,可以用来在训练过程中实时评估模型的性能,无需单独划分验证集。

其次,在构建树的每个节点进行分裂时,不是从所有特征中选择最优特征,而是先从一个随机选取的特征子集中选择最优特征进行分裂。这进一步增强了树的多样性,确保每棵树都学习数据的不同侧面。

单棵决策树非常容易过拟合,通过随机性,每棵树都变得“片面”和“简单”,但众多不同的“片面”树组合起来,反而能捕捉到真正的通用规律,泛化能力极强。同时,由于随机采样,存在的噪音不会对所有树造成影响,整个森林稳定性提升。

2. 通过两个随机操作构建多棵决策树后,使用投票机制:

将新特征输入到森林中的每一棵决策树中,统计每一棵树的输出结果。如果是预测(分类)问题,最终预测结果为得票数最多的那个类别。如果是回归问题,最终结果为所有树输出的结果的平均值。

主要优点:^[2]

随机性可确保单个树之间的相关性较低,从而减少偏差风险。

大量树的存在也减少了过拟合问题。

可以处理缺失值并保持高准确性。

其算法可用于从训练数据集中识别非常重要的特征。

可以处理包含数千个输入变量的庞大数据集,因此成为降维的不错工具。

1.2 SVC支持向量分类机：

SVM 是一种监督学习算法的总称，核心思想是：寻找一个超平面，最大化不同类别之间的间隔。SVC 是 SVM 用于分类问题的具体实现，SVR 是 SVM 用于回归问题的具体实现。游戏情形下，我们选用 SVC。

线性可分与硬间隔：

如果这些训练数据是线性可分的，可以选择分离两类数据的两个平行超平面，使得它们之间的距离尽可能大。在这两个超平面范围内的区域称为“间隔”，最大间隔超平面是位于它们正中间的超平面。^[3]

最大间隔超平面完全是由最靠近它的那些向量确定的。这些向量叫做支持向量。^[3]

硬间隔严格要求所有点都满足约束，没有例外。

线性不可分与核技巧：

但是现实世界的数据往往不是线性可分的。SVM 通过一个非常巧妙的“核技巧”来解决这个问题。基本思想是，如果数据在原始空间（低维）中线性不可分，我们可以将它映射到一个更高维的特征空间。在这个高维空间中，数据可能就变得线性可分了。

核方法之所以得名，是因为使用了内核函数，这使它们能够在高维的隐式特征空间中运行，而无需计算该空间中数据的坐标，而是简单地计算特征空间中所有数据对的图像之间的内积。此作的计算成本通常比显式计算坐标便宜。^[4]

处理噪音与软间隔：

即使使用了核函数，数据中也可能存在一些重叠或噪声点，使得数据并非完美可分。如果严格要求所有点都满足硬间隔，模型可能会过拟合。为此，SVM 引入了软间隔概念。

核心：允许一些数据点“犯错误”，即允许它们落在边际之内，甚至被错误分类。

实现：在优化目标中引入松弛变量 ξ_i 和惩罚参数 C 。

主要优点：

有效处理高维数据

内存效率高

1.3 神经网络-多层感知器：

多层感知器是神经网络中最基础、最重要的模型之一。它是一种前馈人工神经网络^[5]，包含一个输入层、一个或多个隐藏层和一个输出层。除了输入层，每一层都由多个神经元（或称为“节点”、“单元”）组成，并且层与层之间是全连接的。

核心组成部分：

1、层结构，包括输入层负责接收原始数据，隐藏层负责进行特征抽象和复杂变换和输出层产生最终的预测结果。

2、神经元，将来自上一层所有神经元的输入，乘以对应的权重，再加上一个偏置项，得到结果 z ，输入到一个非线性激活函数 f 中，得到该神经元的最终输出。

3、激活函数，是 MLP 能够学习非线性关系的关键。如果没有激活函数，无论有多少层，MLP 都只是等价于一个线性回归模型。

工作过程——前向传播：

- 1、输入数据被送入输入层。
- 2、数据从输入层流向第一个隐藏层。每个隐藏层神经元计算其输入的加权和，然后应用激活函数，得到输出。
- 3、上一层的输出作为下一层的输入，重复步骤 2，直到数据到达输出层。
- 4、输出层产生最终的预测结果。

学习过程——反向传播和梯度下降：

- 1、前向传播：输入一个样本，得到预测输出。
- 2、计算损失：使用损失函数（如均方误差 MSE、交叉熵损失）比较预测输出和真实标签，得到一个误差值。
- 3、反向传播：将这个误差从输出层开始，反向传播回网络中的每一层，并根据链式法则计算出每一层权重和偏置的梯度。
- 4、更新参数：参数沿着梯度的反方向，即减小损失的方向进行微调。
- 5、重复以上过程直到模型收敛。

主要优点：

- 能够学习非线性关系
- 突破了单层模型的限制，具备了强大的函数拟合能力

1.4 梯度提升——xgboost：

梯度提升，亦称作梯度增强，是一种用于回归和分类问题的机器学习技术。其产生的预测模型是弱预测模型的集成^[6]，如深度很小的决策树，来创建一个强大的模型。通过迭代的方式，每一步都构建一个新的模型来修正前一步所有模型组合的残差。

工作原理——三步迭代法：

- 1、初始化模型，用一个非常简单的模型来起步，通常是一个常数。
- 2、迭代构建，对于当前模型和训练集中的每一个样本，计算损失函数的负梯度，即伪残差。再训练一棵新的决策树，去拟合上一步计算出的伪残差。将新训练的树与之前的模型加权组合起来，形成一个新的、更强大的模型。
- 3、重复第二步，直到 M 轮迭代后，将得到最终的强学习器，它是初始模型与所有弱学习器加权之和。

主要优点：

- 预测精度高
- 灵活性高，可以处理各种类型的数据

2 不同模型性能对比

2.1 手动玩法：

- 1、首先操控人物左右移动，适当清除部分障碍物，等待 Aliens 增加聚集。

2、接着目测 Aliens 数量和聚集程度，移动人物至一个中部位置，持续向上发射。

3、此时因为 Aliens 较为聚集，发射命中率较高，不用过多注意是否真正命中 Aliens，只需注意人物头顶有没有 Aliens 投下炸弹，如果有，则及时左右移动避开。

4、直到 Aliens 较为稀疏时，即大约只剩 5-6 个时，此时持续按向上发射键命中效率较低，需计算精确打击时间。

5、观察发现此时只需移动人物至左数第 9 个方格，在某 Alien 左移达到左边缘即将转向时按下向上发射键，人物发射的子弹会正好命中转向右移的 Alien。

6、最后使用 5 中的“精确打击”方法直到所有 Aliens 均被消灭。

2.2 模型性能对比：

人工对比：

分别用四种方法训练出的模型进行游戏，通过人工观察游戏实况（框架代码中给出的前 500 秒），发现不同模型的玩法分别有以下特征：

1.随机森林：只向上射击，不会躲避炸弹，但因为 test 框架代码中游戏只进行前 500 步，Aliens 处于较为聚集的阶段，持续向上射击的命中率较高，因此最终得分较高。

2.SVC 支持向量机：与随机森林差别不大，同样是人物只射击，不会躲避炸弹，最终得分较高。并且开始阶段射击很精准。

3.MLP 多层感知器：人物的主要操作是向上射击，在射击中会有左右移动操作，该操作大部分并没有明显的逻辑，但也出现了少量明确是为了躲避炸弹并成功的操作。同时，左右移动使得其错失部分射击的机会，最终得分较低。

4.梯度提升：会模仿我一开始左右移动并适当清除障碍物的操作，后持续射击，射击过程中存在左右移动操作，但不会躲避炸弹，可能是在模仿我操作时多站在障碍物旁边射击的习惯，最终得分中等。

（有个猜测，或许是持续向上射击且不左右移动，导致人物位置上方 Aliens “浓度”降低，投下炸弹的概率稍微减小，经常一局下来并不需要躲炸弹操作。但这种操作更多靠运气，要提升游戏水平还是应该提升模型对人物头顶上炸弹的关注度，并及时躲避。）

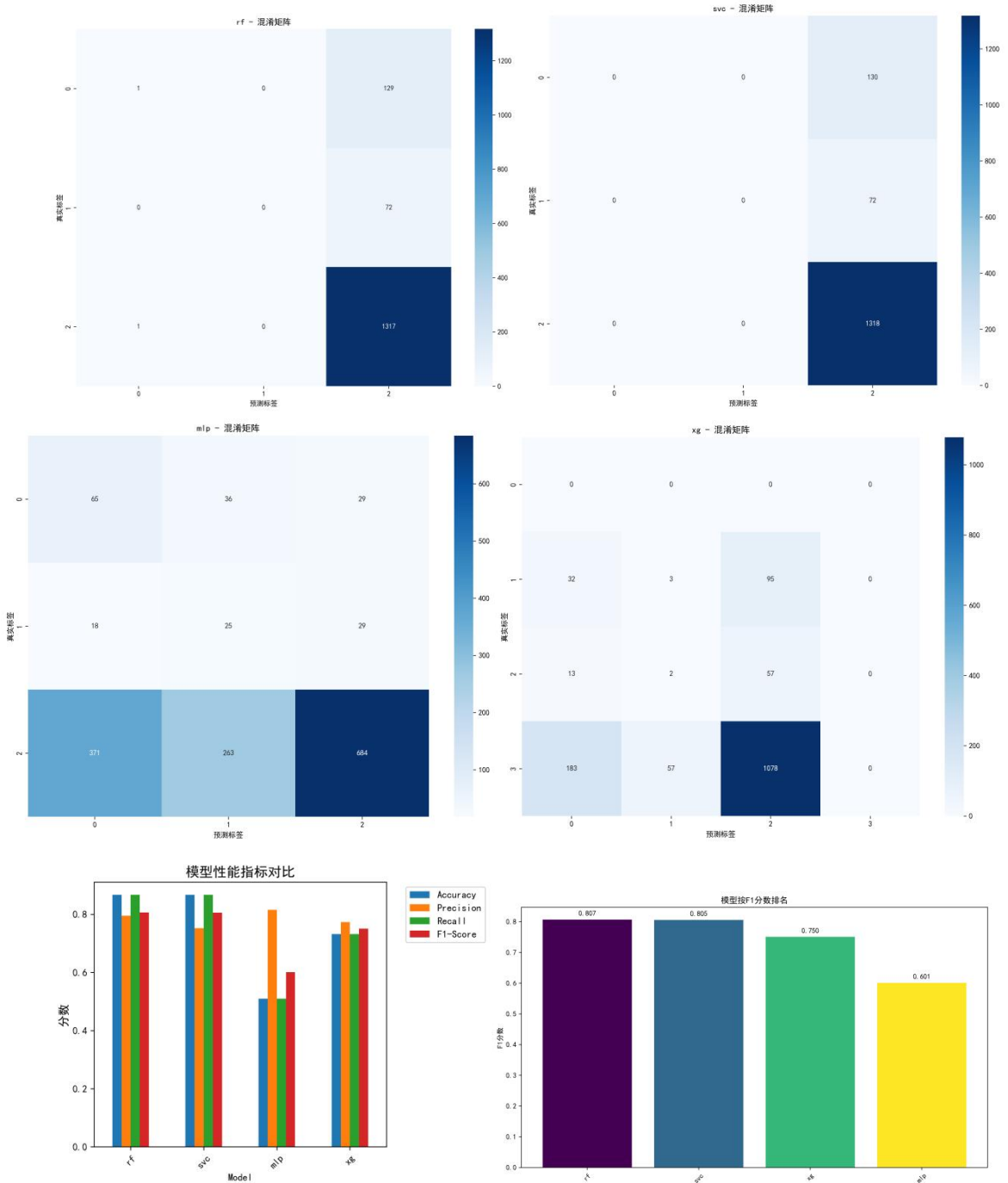
自动对比：

通过查询资料，我了解到用于评估机器学习模型性能的常用指标包括准确率、精确率、召回率、F1 值、ROC 曲线的 AUC 值，而这些评价指标直接或间接都与工具混淆矩阵相关。^[7] Scikit-learn 库中提供了相关的函数和 API 接口：

```
accuracy_score    # 准确率：正确预测的比例
precision_score   # 精确率：预测为正例中真正正例的比例
recall_score      # 召回率：真正正例中被预测出来的比例
f1_score          # F1 分数：精确率和召回率的调和平均
roc_curve         # ROC 曲线：计算真阳性率和假阳性率
auc               # AUC 值：ROC 曲线下的面积，衡量分类器整体性能
confusion_matrix  # 混淆矩阵：显示 TP, FP, FN, TN
classification_report # 分类报告：包含精确率、召回率、F1 等的详细报告
(本游戏为多分类问题非二分类问题，故 ROC 曲线不适用)
```

我阅读源代码发现，lv11 中使用了 portalFast，而其余关卡中使用了 portalSlow，个人认为这二者的区别对操作的判断不会产生影响，但特征提取函数中区分了这二者，我因此用 lv10 的游戏记录作为训练集，lv12-lv14 的游戏记录作为测试集，得到以下结果：

（lv11 作为测试集的结果存放在“模型对比图表（lv11）”中，不在此处展示了，大致和下图差别不大。下图可在“模型对比图表（slow）”中查看原图。）



可观察到 random-forest 和 svc 四个评价指标得分均很高，混淆矩阵中非对角线元素较少，xgboost 得分紧跟其后，而 mlp 得分较低。

但事实是得分最低的 mlp 是我唯一一个观察到有躲避炸弹操作的模型，而得分高的两个模型基本只会射击。

我认为这说明了用不同评价标准来看四个模型得到的结果不同，从得分数据的角度来说，评价指标的比较结果和游戏得分的比较更加吻合；而从行为模仿的角度来说，四个模型分别表现出对我的不同行为的模仿，四个模型均模仿了持续射击操作，同时 svc 可能存在模仿看准位置射击的行为，mlp 还存在模仿躲避炸弹的行为，xgboost 还存在模仿适当清理障碍物的行为，侧重不同，无法用单一标准完全概括孰优孰劣。

3 改进特征提取方法

3.1 原功能及其不足之处：

原功能仅记录屏幕上每个位置的信息，并将每一个单元格均存成一个九维向量，九个维度分别对应九种元素是否存在于该单元格中。

则每个特征向量维度：网格行数 $\times$ 网格列数 $\times$ 9，对应一个操作代码。

我认为存在以下不足之处：

- 1、维度过大，占用过多内存和计算资源，且容易过拟合
- 2、部分数据，比如地板墙壁门之类的，几乎不影响决策，放入数据集中会产生噪声
- 3、所有元素没有权重差别，没有突出 Aliens 数量和躲避炸弹的重要性

3.2 改进之处：

减少维度和提取的元素种类，给每个元素分配权重。

提取的信息包括：

- 1、所有 Aliens 的位置
- 2、avatar 的位置
- 3、avatar 所在格包括左右两格的上方炸弹情况
- 4、avatar 所在格包括左右两格的上方障碍物情况
- 5、avatar 左右是否靠墙

相关函数为：

```
def extract_features(observation): 5 用法 Alike
    def get_basic_info():...
    height,width,portal,avatar_x,avatar_y=get_basic_info()
    def extract_alien():...
    def extract_bomb():...
    def extract_avatar():...
    def extract_base():...
    def extract_wall():...
    features = np.concatenate([
        extract_alien(),
        extract_avatar(),
        extract_bomb(),
        extract_base(),
        extract_wall()
    ])
    return features
```

权重调整：

- 1、炸弹的重要性最大，设为 100（原先设为 10，但效果提升并不明显）
- 2、Aliens 权重为 5，avatar 权重为 3，希望模型能够建立二者位置联系再决定射击时机和站位。
- 3、其他元素，墙和障碍物，较不重要，保持权重为 1。

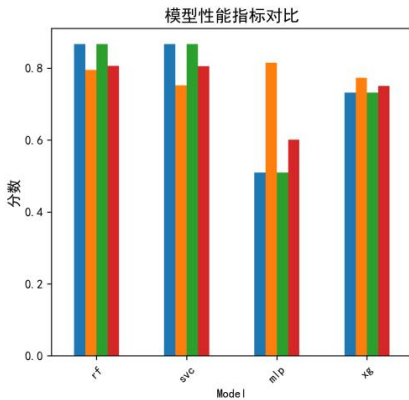
3.3 改进后模型对比：

人工对比：

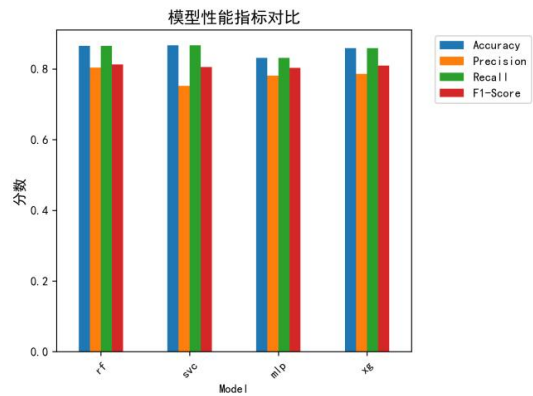
较改进前：

- 1.**随机森林**：在向上射击的基础上加入了一些游戏初期左右移动适当清理障碍物的操作，出现了少量躲避炸弹的操作，最终得分较改进前增加。
 - 2.**SVC 支持向量机**：与改进前差别不大，同样是人物只射击，不会躲避炸弹。（这点在后文的混淆矩阵中也有所体现，预测标签只有射击一种操作）
 - 3.**MLP 多层感知器**：同样增加了清理障碍物的操作，但与随机森林相比逻辑较显混乱；躲避炸弹能力有所提升，大部分情况能够出现明显的躲避或远离操作，但也有少次躲避失败。
 - 4.**梯度提升**：增加了躲避炸弹的操作，但是较 MLP 效果稍弱，躲避成功的情况较少。
- （另外，若延长游戏时间，发现游戏后期四种模型仍没能学会精准射击，猜测或许和数据采集时没有收集操作 0（不动）的情况有关，模型认为每一步必须有左移右移或射击操作，则缺少不动等候时机的操作，但既然这里原代码只看前 500 步，就不过多展开研究了）

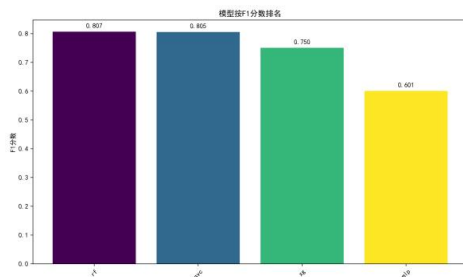
自动对比：



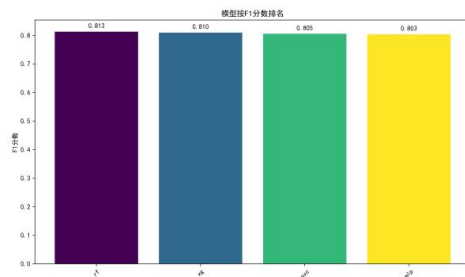
原：



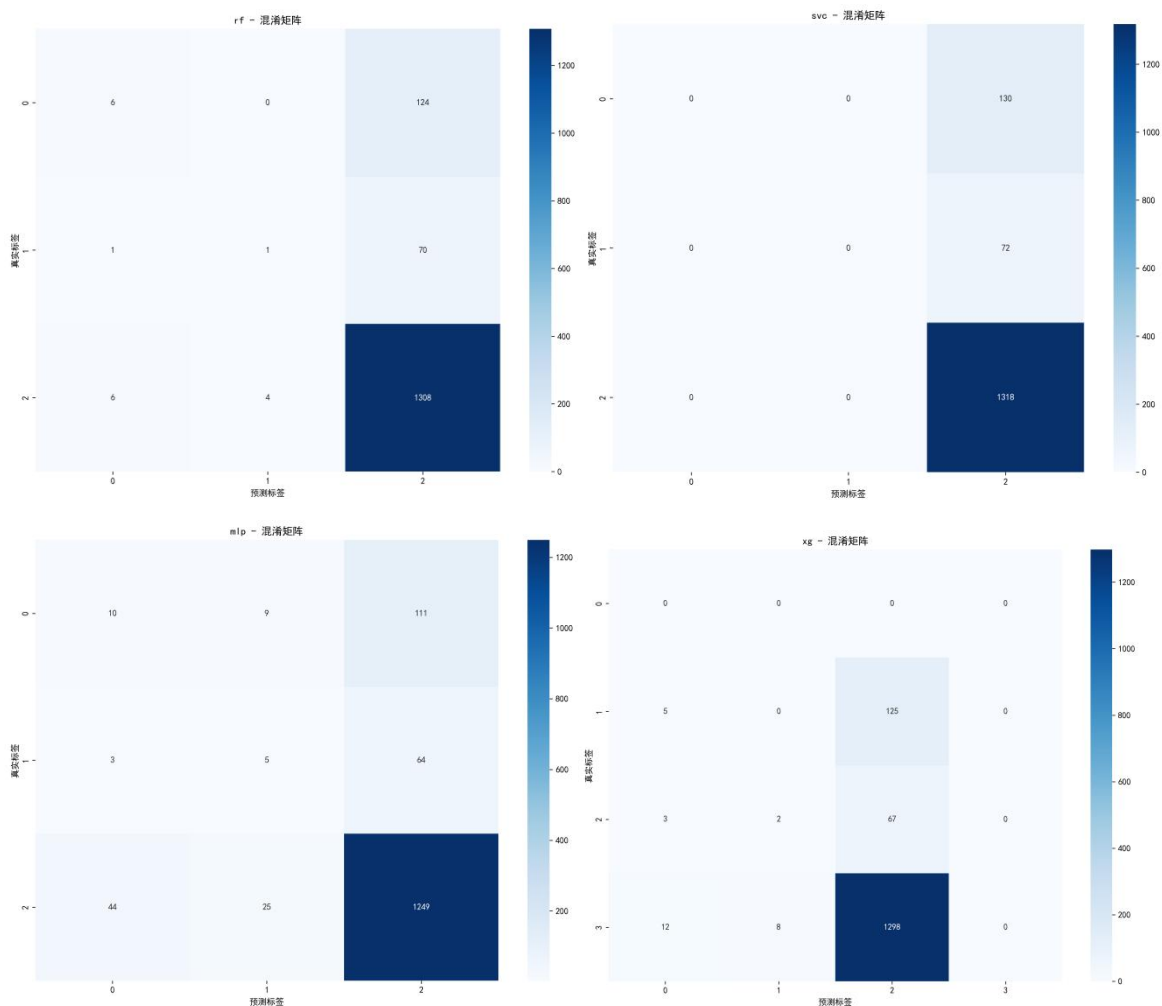
现：



原：



现：



可观察到较改进前，mlp 和 xgboost 的性能指标有了明显的提升，目前四种模型的差距缩小了。这点与观察游戏实况得到的结果一致，mlp 和 xgboost 已学会清除障碍物和躲避炸弹两种游戏策略，是性能较好的模型。

4 reference

[1] [随机森林 - 维基百科](https://en.wikipedia.org/wiki/Random_forest) (https://en.wikipedia.org/wiki/Random_forest)

[2] [什么是随机森林？ | 数据科学 | NVIDIA 术语表](https://www.nvidia.cn/glossary/random-forest/) (https://www.nvidia.cn/glossary/random-forest/)

[3] [支持向量机 - 维基百科，自由的百科全书](https://zh.wikipedia.org/wiki/%E6%94%AF%E6%8C%81%E5%90%91%E9%87%8F%E6%9C%BA%E7%BA%BF%E6%80%A7SVM)

(https://zh.wikipedia.org/wiki/%E6%94%AF%E6%8C%81%E5%90%91%E9%87%8F%E6%9C%BA%E7%BA%BF%E6%80%A7SVM)

[4] [内核方法 - 维基百科](https://en.wikipedia.org/wiki/Kernel_method) (https://en.wikipedia.org/wiki/Kernel_method)

[5] [多层感知器 - 维基百科](https://en.wikipedia.org/wiki/Multilayer_perceptron) (https://en.wikipedia.org/wiki/Multilayer_perceptron)

[6] [梯度提升技术 - 维基百科，自由的百科全书](https://zh.wikipedia.org/wiki/%E6%A2%AF%E5%BA%A6%E6%8F%90%E5%8D%87%E6%8A%80%E6%9C%AF)

(https://zh.wikipedia.org/wiki/%E6%A2%AF%E5%BA%A6%E6%8F%90%E5%8D%87%E6%8A%80%E6%9C%AF)

[7] [混淆矩阵、准确率、召回率、F1 值、ROC 曲线的 AUC 值 - 知乎](https://zhuanlan.zhihu.com/p/356457537) (https://zhuanlan.zhihu.com/p/356457537)